

SKILL WEEK - 4

Task 1

Code :-

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int tokenize(char *input, char *tokens[])
{
    int count =
0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS - 1)
    {
        tokens[count++] = token;    token
= strtok(NULL, " \\t\\n");
    }

    tokens[count] = NULL;    return
count;
}

int validate_tokens(char *tokens[], int count)
{
    if (count ==
0)
    {
        printf("Empty command\\n");
return 0;
    }

    for (int i = 0; i < count; i++)
    {
        if (strcmp(tokens[i], "|") == 0 &&
            (i == 0 || i == count - 1))
        {
            printf("Syntax Error: Invalid pipe position\\n");
return 0;
        }
    }
}
```

```

    return 1;
}

void display_tokens(char *tokens[], int count)
{
    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }
}

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];

    while (1)
    {
        printf("\nmyshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)        break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)        break;

        int count = tokenize(input, tokens);

        if (!validate_tokens(tokens, count))
            continue;

        display_tokens(tokens, count);
    }

    return 0;
}

```

OUTPUT:-

```

avulasanjana@SanjanaReddy:~$ cd ~/OSSP
avulasanjana@SanjanaReddy:~/OSSP$ ls
OSSP  src
avulasanjana@SanjanaReddy:~/OSSP$ nano parser.c
avulasanjana@SanjanaReddy:~/OSSP$ gcc parser.c -o parser
avulasanjana@SanjanaReddy:~/OSSP$ ./parser

myshell> ls -l

Tokens:
Token 1: ls
Token 2: -l

myshell> ls -l /home

Tokens:
Token 1: ls
Token 2: -l
Token 3: /home

myshell>
Empty command

myshell> exit

```

Task 2:

Code:- #include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#define MAX_INPUT 1024

#define MAX_ARGS 100

int parse_command(char *input, char *args[])

{

int count = 0;

char *p = input;

while (*p != '\0')

{

// Skip spaces while (*p

```
== ' ' || *p == '\t')    p++;
```

```
    if (*p == '\0')  
break;
```

```
    // Single quoted string    if  
(*p == '\')  
    {  
p++;
```

```
    args[count] = p;
```

```
    while (*p != '"' && *p != '\0')  
p++;
```

```
    // Unmatched quote    if  
(*p == '\0')  
    {  
        printf("\nSyntax Error: Unmatched single quote.\n");  
return -1;  
    }
```

```
    *p = '\0';  
p++;
```

```
    count++;  
}    else  
{  
    // Normal argument  
args[count] = p;
```

```
    while (*p != ' ' && *p != '\t' && *p != '\0')  
p++;
```

```
    if (*p != '\0')  
    {  
        *p = '\0';    p++;  
    }
```

```
    count++;  
}  
}
```

```
    args[count] = NULL;    return  
count;
```

```

}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("\nmyShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)        break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)        break;

        int count = parse_command(input, args);

        if (count == -1)
        continue;

        if (count == 0)
        continue;

        printf("\nParsing Result:\n");

        for (int i = 0; i < count; i++)
        {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            continue;
        }
    }
}

```

```

        if (pid == 0)
        {
            execvp(args[0], args);

            perror("execvp");
        }
        exit(1);
    }
    else
    {
        printf("\nChild PID: %d\n", pid);

        waitpid(pid, NULL, 0);

        printf("\nCommand execution completed.\n");
    }
}

return 0; }

```

OUTPUT:-

```

avulasanjana@SanjanaReddy:~$ nano parser_quotes.c
avulasanjana@SanjanaReddy:~$ gcc parser_quotes.c -o parser_quotes
avulasanjana@SanjanaReddy:~$ ./parser_quotes

myShell> echo Hello
Parsing Result:
Argument 1: echo
Argument 2: Hello
Child PID: 668
Hello
Command execution completed.
myShell> echo 'Hello World'
Parsing Result:
Argument 1: echo
Argument 2: Hello World
Child PID: 669
Hello World
Command execution completed.
myShell> echo '$HOME'
Parsing Result:
Argument 1: echo
Argument 2: $HOME
Child PID: 672
$HOME
Command execution completed.
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.

```

```
myShell> echo 'Operating System'
```

```
Parsing Result:
```

```
Argument 1: echo
```

```
Argument 2: Operating System
```

```
Child PID: 689
```

```
Operating System
```

```
Command execution completed.
```

```
myShell> exit
```

Task3 :-

Code:-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_INPUT 1024
```

```
#define MAX_ARGS 100
```

```
int parse_command(char *input, char *args[])
```

```
{
```

```
    int count = 0;
```

```
    char *p = input;
```

```
    while (*p != '\0')
```

```
    {
```

```
        // Skip spaces
```

```
        while (*p == ' ' || *p == '\t')    p++;
```

```
        if (*p == '\0')
```

```
        break;
```

```

        // Double quoted argument      if
(*p == '"')
    {
        p++;      args[count]
= p;

        while (*p != '"' && *p != '\0')
p++;
        // Check unmatched double quote
        if (*p == '\0')
        {
            printf("\nSyntax Error: Unmatched double quote.\n");
return -1;
        }

        *p = '\0';
p++;

        count++;
    }      else
    {
        // Normal argument
args[count] = p;

        while (*p != ' ' && *p != '\t' &&
*p != '"' && *p != '\0')      p++;

        if (*p == '"')
        {
            printf("\nSyntax Error: Unexpected double quote.\n");
return -1;
        }      if
(*p != '\0')
        {
            *p = '\0';      p++;
        }

        count++;
    }

```



```

    }

    args[count] = NULL;
return count;
}

/* Expand $USER and other environment variables */ void
expand_variables(char *args[], int count)
{
    for (int i = 0; i < count;
i++)
    {
        char result[MAX_INPUT] = "";
        char
        *p = args[i];

        while (*p != '\0')
        {
            if (*p
== '$')
            {
                char variable[100];
                int
j = 0;

                p++;

                while ((*p >= 'A' && *p <= 'Z') ||
(*p >= 'a' && *p <= 'z') ||
                (*p >= '0' && *p <= '9') ||
                *p == '_')
                {
                    variable[j++] = *p;
p++;
                }

                variable[j] = '\0';

                char *value = getenv(variable);

                if (value != NULL)
strcat(result, value);
            }
        }
    }
}

```

```

else
    {
        int len = strlen(result);
        result[len] = *p;        result[len
+ 1] = '\0';        p++;
    }
}

strcpy(args[i], result);
}
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("\nmyShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        int count = parse_command(input, args);

        if (count == -1)
            continue;

```

```

    if (count == 0)
        continue;

    // Allocate buffers for variable expansion
    char expanded[MAX_ARGS][MAX_INPUT];

    for (int i = 0; i < count; i++)
    {
        strcpy(expanded[i], args[i]);
args[i] = expanded[i];
    }
    // Expand environment variables    expand_variables(args,
count);

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < count; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
continue;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
printf("PID = %d\n\n", getpid());

        execvp(args[0], args);

        perror("execvp");
exit(1);    }    else
    {

```

```
waitpid(pid, NULL, 0);

printf("\nChild process
completed.\n");
}
}

return 0; }
```

Ouput:-

```
avulasanjana@SanjanaReddy:~$ nano parser_double_quotes.c
avulasanjana@SanjanaReddy:~$ gcc parser_double_quotes.c -o parser_double_quotes
avulasanjana@SanjanaReddy:~$ ./parser_double_quotes

myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello avulasanjana

Child Process
PID = 850

Hello avulasanjana

Child process completed.

myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 851

Hello World

Child process completed.

myShell> echo "Hello World

Syntax Error: Unmatched double quote.

myShell> "My name is $USER"

--- Parsed Arguments ---
Argument 1: My name is avulasanjana

Child Process
PID = 854

execvp: No such file or directory

Child process completed.
```